Maruf Sir

14.6.26

Compiler: a software/program that converts high level lang to low level lang/ machine lang

⇒ Compiler takes High level programming lang as input

⇒ " gives output Low level "

High Level Programming Lang } source code

↓

Compiler

↓

Low level Programming Lang } Target code

Assembly Lang/Machine Lang

⇒ C program IDE → Codeblocks, IDE and Compiler not same

যখন কোনো software develop করি তখন তেই development related যে যে ধরনের tools for

debug, simulate, execute এগুলোকে একত্রে IDE

↓

Integrated Development Environment

Compiler: MinGW works for convert only from H·L·L to L·L·L

C program (H·L·L)

↓

MinGW (Compiler)

↓

machine lang. (L·L·L)

কম্পিউটার বুঝতে পারে Language

Processing system / Language processor

H·L·L / Source code

```
#include <stdio.h>
int main()
{ int a,b,sum;
  a=10;
  b=20;
  sum = a+b;
  printf("Sum = %d", sum);
  return 0;
}
```

Preprocessor directive

→ Preprocessor

Source code এর যেখানে #include<stdio.h> কে দেখে H·L·L কে pure H·L·L এ convert করে। যুক্তে define থাকলে code এ replace করে দেয় (e.g. π = 3.14)

↓ → Pure H·L·L

Compiler — 6 টা অপারেশন step

↓ → Assembly lang

Assembler

↓ → Relocable machine code

Linker/Loader

↓

101100 01101 — Machine Language / Target code

Linker: একাধিক source file / library funct কে একটা object code এ convert করে

Loader: secondary storage এ যে program টা আছে সেটা linker ready করে দেয়, সেটাকে ~~process~~ RAM এ নিয়ে যায়

Language Processing System: is a set of softwares/ tools that converts H.L.L to L.L.L

Compiler → reads whole program (source code) then shows error

Interpreter → reads line by line and execute and shows error

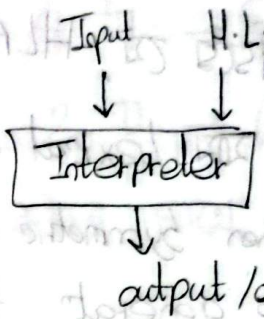
⇒ immediate output

→ error থাকলে আবার নতুন করে interpret করে

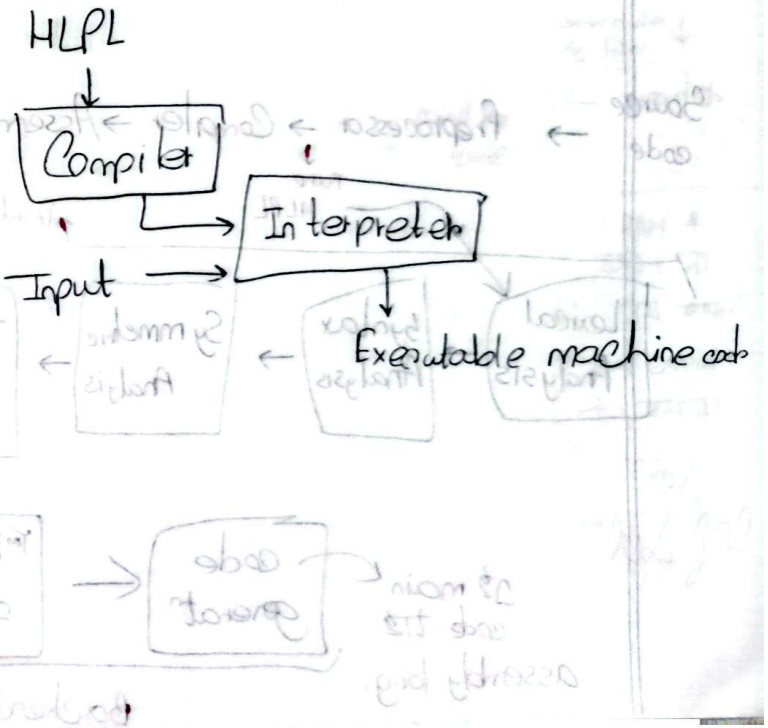
→ But for compiler একবার compile হয়ে গেলে দুই বোনক আর করেনা, Compiler is faster

⇒ Java has both interpreter and compiler একইসাথে interpreter and compiler দুই কাজ করে। So Java is slow

Interpreter



For Java



Compiler

Preprocessor ছাড়া কম্পাইলার যে, HLPL টা প্রসেস হোক first এ যায় Lexical analysis then Syntax analysis then Symmetric analysis then intermediate code generatn then code optimizatn then target code generatn

Source code → Preprocessor → Compiler → Assembler → Linker/Loader → Target code

Pure HLPL

Front end phase

Lexical Analysis → Syntax Analysis → Symmetric Analysis → Intermediate code generatn → Target code optimizatn → code generatn

Compiler 6 steps

সে main code টা assembly lang.

Backend phase

প্রথম 4 টা → Frontend phase

পরের 2 টা → Backend phase

প্রত্যেক টার সাথে error handling থাকে

Source code → preprocessor → Compiler (Pure HL.L.) → assembler (Assembly lang) → Linker (Machine code/object code)

Linker combines diff obj files → Loader → machine code

RAM এ যাওয়ার না নিয়ে আসার real execution টা হয় না

Preprocessor → Comment করা এবং বাদ করে অনেক সময় Lexical analyzer ও এই কাজ টা করে depends on compiler

→ Preprocessor এর কাজ → Imp

#include<stdio.h> এর মধ্যে prototype টা যেটাকে add করে দেয় program এ

Exam →

Compiler structure → CT তে আসতে পারে

6 টি step, slide এ আছে

Language processing system এর structure → "

21.6.26

→ তবে compiler এর ক্ষেত্রে Lexical to intermediate code generat same, last 2 টি step depend করে কোন OS

→ LPS এর সবথেকে important part compiler

Compiler এর main funct slide এ given

→ Lexer/Scanner/Lexical analyser transforms the programs into tokens

→ Syntax analyzer/Parser checks grammar of program

→ Semantic analyser " meaning of program

int a syntax correct

a = "Hello" int হিসাবে নাবে, string হিসাবে

→ Intermediate code generator converts source code into intermediate form

→ Code optimization improves the code

c = a+b

a+b এর value যোগ করে যোগফল টা দিয়ে আমি যেন compiler এর thinking করা করা লাগে

⇒ Code generat" produces target code

⇒ Error handling

⇒ Symbol Table management program এ যে identifier আছে সেগুলো থাকে symbol table

Common for all phases of compiler

⇒ Compiler এর প্রত্যেকটা phase এ error handling and symbol table management থাকে

⇒ Identifier → variable names

⇒ Symbol Table একটা data structure যেখানে identifier, এর info রাখি specific কাজের জন্য যেকোন programming lang এর প্রোগ্রাম linear DS দিয়ে symbol table generate করা যায়

→ Java এ সেজন্য hash table/hash map দিয়ে symbol table generat" এর জন্য

Symbol table funct"

⇒ Identifier এর funct", scope, address hold করে

⇒ Lexical analysis stage এ symbol table generate হয়, বাকি সব stage এ symbol table modify হয়। No of rows, columns বাড়তে থাকে। Lexical analysis stage এ identifier create হলে সেটা symbol table এ first entry করবে

⇒ Assembler creates obj code but the program is not always ready to run yet এখানো linking এর কাজ বাকি

⇒ Loader functions

Lab → symbol table, location

Lexical analysis

int a = 10; → প্রত্যেকটা character নেয়

Lexer প্রত্যেকটা line কে sequence of character হিসেবে input হিসেবে নেয় and produce token

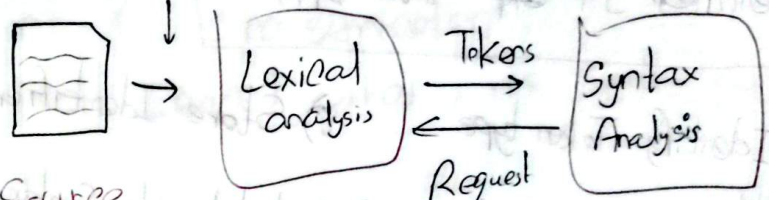
⇒ Source code কে string of char হিসেবে নেয়

⇒ source code এর কোনটা variable/funct ভুল দেখেনা

⇒ space আসলে বুঝে নেয় যে একটা word শেষ হয়েছে

i n t space ভুল বুঝে নেয় যে int একটা word. তার পর দেখে এই word টা কি variable নাকি identifier নাকি funct

Pure HL PL



Exam

Draw this diagram for Lexical analysis

কী ? কীভাবে কাজ করে ? → for short ques

For Lexical analysis এ অনেক ভাগ আছে → for broad ques

objectives of Lexical analysis

1) Read source code
2) Removes unnecessary space and comment
   → যদি preprocessor কাজ না করে comment গুলো lexer করে দেয়
3) Grp char into Lexemes
   → meaningful part of a program
4) Convert Lexems into Token

Next Class DFA, NFA, reg. exp

Identifier এর ~~exp~~ pattern into DFA

5) Identify Token type → 6) Store Identifier

6) Lexical errors 8) Send token to Syntax analyzer

Exam এ part of code দেওয়া থাকবে find lexical errors / syntax errors Code এ অনেক error থাকবে but সবই error তো lexical এ পড়বে না

Ex: i) int main) (

এটা lexical error না, এখানে token generate করে দেবে, এটা syntax error

ii) 2 marut

এটা identifier এর pattern follow করেনি lexical error

iii) int X@Y

Lexical error, no special char

iv) int X no semicolon.

Not Lexical error, it's syntax error

v) b++++

token token token

operator

b = variable

++ = operator

Not lexical error

Terms

Character: Smallest unit of source code

Lexeme: char গুলো কে একসাথে meaningful part হিসেবে ধরে নেয় space এর আগের part টা কে

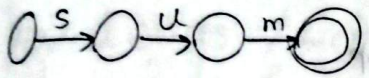
Token: lexeme গুলো similar type করে একসাথে নিয়ে আসি, সেই category কে token বলে

Ex: Numbers, constants

Pattern: rule to identify token

Symbol taken: identifier এর information hold করে

⇒ Lexical analysis stage এ Match lexemes with token pattern এটা most important stage



এরকম match করাবে যে এটা keyword নাকি variable

→ Space DFA তে থাকবে না

⇒ meaningful word এর lexeme বের করে DFA

Exam ☆ How lexical analysis works at a glance 7/8 marks এর জন্য আসতে পারে draw the fig. given in slide

Exam ☆☆

```
int sum (int a, int b)
{ int c;
  c = a+b;
  return c;
}
```

⇒ Find tokens and count

Ans: Divide each line in rows

	Tokens
int sum (int a, int b)	9
{ int c;	4
c = a+b;	6
return c;	3
}	1
	= <23

Token count করতে invalid valid both

⇒ যদি exam এ int sum) int a, int b(এরকম syntax wrong হলেও token হিসাবে count হবে

c = a+b++ ?

Exam

যদি x_@_y → Valid token int, x, ..., y @ কে ধরবে না

Exam 1 কয়টা valid token

int sum) int a, int b (

{ int x@y → This is not valid token

c = a+b বাকিগুলো valid

return c;

}

⇒ find token errors // lexical errors

⇒ Categorize token কতগুলো identifier, keyword

⇒ int x @ y; এখানে 4 টা token

i) >= token

ii) +++ 2 token

iii) +-+ 3 token

iv) => 2 token

Valid হলে কোনো token হিসেবে নিবে কেন

invalid ,, individual token হিসেবে নেবে

11XYZ Token error

"XYZ" পুরাটা একটা token

Identifier rules → (a-z) ∪ (A-Z) ∪ _ ∪ (0-9) underscore

⇒ (a-z) any letter

⇒ underscore হতে পারে তারপর star

⇒ Rafi 1 valid, ra_fi valid

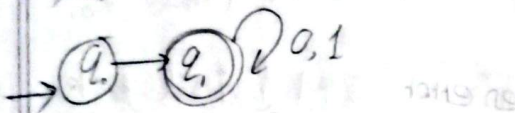
⇒ 1 Rafi invalid → no দিয়ে শুরু হবেনা

⇒ No special char → @ x

⇒ _stu valid

Finite automata 5 tuple

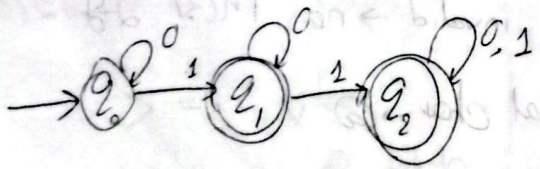
1) Q = set of states $\{q_0, q_1\}$

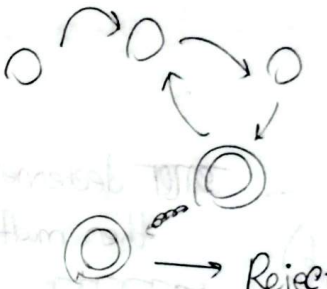


2) Σ = inputs = $\{0, 1\}$

3) δ:

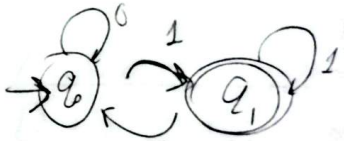
Current state	Input	Next state
q_0	0	q_0
	1	q_1
q_1	0, 1	q_0, q_1





→ Reject state, এখানে কোন পৌঁছায়নি

Set of binary strings that ends with '1'



$q_0 \xrightarrow{0} q_0$ 1
$q_0 \xrightarrow{1} q_1$ 0
$q_1 \xrightarrow{1} q_1$ 1
$q_1 \xrightarrow{0} q_0$ 1
0
1

⇒ 1 দিয়ে শেষ হলে accepted state

~~rejected~~ accepted state এ পৌঁছায়নি

accept state 1 পৌঁছায়

Example-1 Token finding

```
int main () {
int a=5, b=10, c;
c=a+++--b*2;
if (c>=20 && a!=b)
c+=5;
}
```

→ আগে decrement then multiply হবে

int, main, (,), {	5
int, a, =, 5, ,, b, =, 10, ,, c, ;	11
c, =, a, ++, +, --, b, *, 2, ;	10
if, (, c, >=, 20, &&, a, !=, b,)	10
c, +=, 5, ;	4
}	1

Example-2

```
int main () {
int a = 5;
b = 10;
c;
c = a++ + b
; }
```

int main () {	5
int a = 5;	5
b = 10;	4
c;	2
c = a++ + b	6
; }	2
	23

a=identifier
++=operator
হবে

a++ → total 2 টি token

++ + → 2 টি token

Manuf Sir

5.7.26

Mid + final এ ques আসবে → Presentation থেকে

⇒ Source program remains in secondary storage during execution by compiler Secondary storage is slow. এই process টা কে fast করার জন্য buffer technique use হয়

Compiler RAM এ একটা buffer তৈরি করে Source program কে character এ divide করে buffer এ store করে then বারবার operation হয়, বারবার secondary storage এ যেতে হয় না. This is called Input buffer in lexical analysis

Draw the fig / Fig টার explanation লাগবে

Lexeme begin

↓

int ☐ x ☐ i n ;

slide এ 'n' থেকে দেখানো, এটা ';' থেকে

হবে lexeme begin

দুইটা pointer এর কাজ কী

1) lexeme begin

2) forward pointer → one by one character এগোবে

Exam ★★

⇒ What is Sentinal, lexeme begin, forward pointer

⇒ Why 2 buffer?

Faster হয়

Transition diagram → একটা visualisation / flowchart যেখানে pattern identify হয়

Maruf Sir

Syntax analysis

19.8.26

Lexical error

sum = 2a + b ✗ → identifier এর শুরু তে number থাকবে না

sum = a + b ✓

sum = +ab ✓

↓

Syntax error

↙ Steps of Syntax analysis

Teacher (noun) is (verb) fantastic (adj)

Students (noun) are (verb) awsome (adj)

Students (noun) are (verb) awsome (adj)

verb phrase

Sentence

Production Rules

S → <Noun> <Verb Ph>

VP → <V> <Adj>

N → Teacher | Students

V → are | is

Adj → fantastic | awsome

Non terminal → S, VP, N, V, Adj

Terminal → Teacher, fantastic, Awsome, Students

P.R → S

Start symbol → S

Grammar, G = {N, T, P, S}

এইরকম grammar এর rules Parser কে চেক করা হয়। যদি sentence এমন আসে। teacher fantastic is তাহলে error দেখাবে

Context free grammar আর Grammar এর diff

1) একাধিক non terminal একসাথে হয়না
2) একটার সাথে আরেকটার dependency নাই
3) Compiler এর জন্য best

Exam ☆☆ Parse tree Exam এ আসবে

Parse Tree 2 types

Top down

Bottom up

LL(1), LL(k) → self study

Exam ☆☆ Grammar Rules

$E \rightarrow E+T \mid T$

$T \rightarrow T*F \mid F$

$F \rightarrow (E) \mid id$

Tapgent String: id + id*id

⇒ Exam এ এরকম Rules and String দেওয়া থাকবে তে left most derivation and right " "

যেকোনো একটা Rules দিয়ে শুরু করতে পারি

{Dont skip any steps}

Left most derivation

$E \rightarrow E+T$ → left থেকে derival শুরু

$\rightarrow T+T$

$\rightarrow F+T$

$\rightarrow id+T$

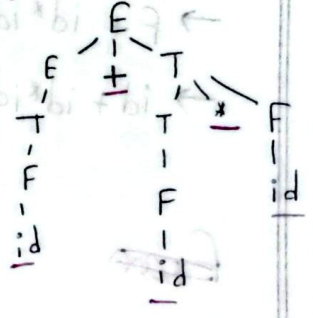
$\rightarrow id+T*F$ → left এর কাজ শেষ তাই right টা কে derive করব

$\rightarrow id+F*F$

$\rightarrow id+id*F$

$\rightarrow id+id*id$

Parse tree



id + id* id

Right to left derivation

Rightmost derivation

$E \rightarrow E+T$

$\rightarrow E+T*F$

$\rightarrow E+T*id$

$\rightarrow E+F*id$

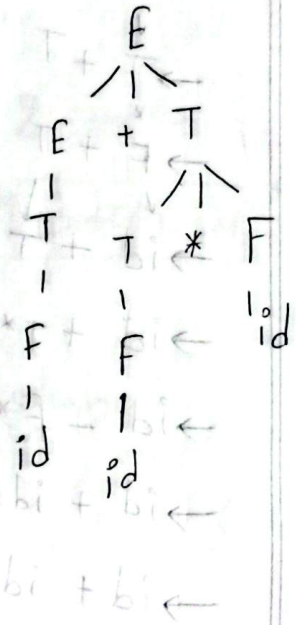
$\rightarrow E+id*id$

$\rightarrow T+id*id$

$\rightarrow F+id*id$

$\rightarrow id+id*id$

Parse tree



Example-2

G.R $S \rightarrow (S)S \mid \varepsilon$

Target string: ()()

Leftmost derivation

$S \rightarrow (S)S$

$\rightarrow ()S$

$\rightarrow ()(S)S$

$\rightarrow ()(S)$

$\rightarrow ()()$

Example-3

G.R : $E \rightarrow E+E \mid E*E \mid id$

T.S. id + id * id

Ambiguous grammar

Leftmost derivat' দিয়ে দুইভাবে Target string পাওয়া যায় but parse tree diff হবে

Leftmost derivat'

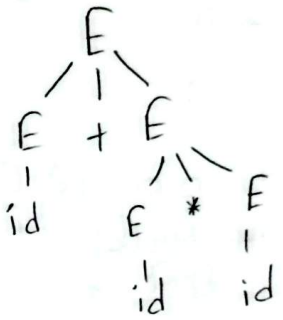
$E \rightarrow E*E$
$\rightarrow E+E*E$
$\rightarrow id+E*E$
$\rightarrow id+id*E$
$\rightarrow id+id*id$

Or,

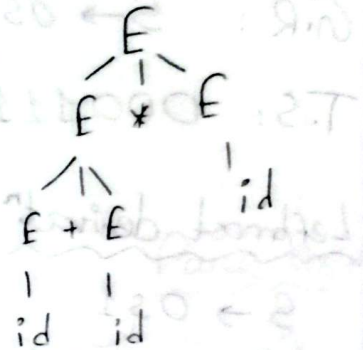
$E \rightarrow E+E$
$\rightarrow id+E$
$\rightarrow id+E*E$
$\rightarrow id+id*E$
$\rightarrow id+id*id$

Exam ☆☆ Is this grammar ambiguous or not

Parse tree



Or,



☆☆ ⇒ In exam show derivat' and parse tree and prove ambiguous or not

Example-4

G.R : $S \rightarrow 0S1 \mid 01$

T.S: 000111

Leftmost derivation

$S \rightarrow 0S1$

$\rightarrow 0\,0S1\,1$

$\rightarrow$ ~~0001 11~~

$\rightarrow 000111$

Example-5

G.R : $S \rightarrow AB$

$A \rightarrow aA \mid a$

$B \rightarrow bB \mid b$

T.S: aaabbb

Leftmost derivation

$S \rightarrow AB$

$\rightarrow aAB$

$\rightarrow aaAB$

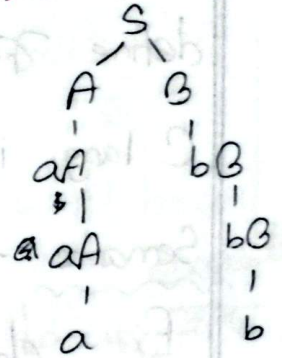
$\rightarrow aaaB$

$\rightarrow aaabB$

$\rightarrow aaabbB$

$\rightarrow aaabbb$

Parse tree



Self Study

⇒ কোন technique follow করলে ambiguous কে non ambiguous এ convert করতে পারি

⇒ precedence কী

⇒ Associativity